

1. Visão do Produto

- **Para** jogadores de PC que querem economizar tempo e dinheiro ao comprar jogos digitais
- **Que** precisam procurar manualmente em vários sites, como Steam, EpicGames para achar jogos que achem interessante e estejam em promoção
- **O** LootChaser é um sistema **que** busca solucionar esse problema se dispondo a rastrear ofertas de jogos, disponibilizar histórico de preços e notificar ofertas
- **Diferente** de outros rastreadores de promoções genéricos e sem alertas personalizados, **nosso produto** combina um sistema simples de rastreamento de dados em tempo real com um motor de notificações, assim o jogador pode adquirir o jogo pelo menor preço e recebe alertas no momento que seu jogo favorito entrar em oferta.

2. MVP & Hipótese de Valor

Acredito que os usuários irão montar suas wishlist's no LootChaser porque irão receber alertas instantâneos no menor preço que o jogo já teve até o momento, sem necessidade de checar em múltiplas lojas virtuais pelo mesmo jogo.

No MVP	Fora do MVP
Autenticação e Perfil	Gráficos do Histórico de Preços
Coleta Automatizada	Extensão para Navegadores
API de Promoções	Integração Direta com Gateways de Pagamentos

3. Backlog do Produto

O acompanhamento através do quadro kanban está configurado aqui:
<https://github.com/users/nattuts/projects/4/views/1> O quadro continuará sendo atualizado

Prioridade	História	Critérios de Aceitação	Sprint
P1	Como jogador, quero listar jogos em promoção para encontrar as melhores ofertas	filtra por faixa de preço; ordenado por maior desconto; paginado	1
P1	Como administrador, quero cadastrar lojas parceiras para ampliar o catálogo	CRUD com o URL da loja validado; loja única por domínio	1
P1	Como sistema, quero coletar ofertas de lugares como Steam periodicamente para manter os preços atualizados	deduplica ofertas por id do jogo	2
P2	Como jogador, quero busca rápida por título do jogo	cache das consultas frequentes ativado; métrica de hit rate	2

Prioridade	História	Critérios de Aceitação	Sprint
P2	Como jogador, quero me autenticar para salvar minha wishlist	JWT com refresh token	3

4. Entidades de Domínio:

- **Game:** representação do jogo ('id', 'titulo', 'data_lancamento', 'capa_url').
- **Store:** representação da loja parceira ('id', 'nome', 'dominio', 'base_url').
- **Ofer:** representação de uma promoção ativa ('id', 'game_id', 'store_id', 'preco_original', 'desconto', 'url', 'atualizado_em').
- **User:** representação do usuário ('id', 'email', 'user_name', 'password_hash', 'criado_em').
- **Wishlist:** representação da conexão entre o usuário e a loja parceira através do jogo ('user_id', 'game_id', 'preco_desejado', 'alerta_notificacoes')

5. Justificativa:

- Optei por Java com Quarkus pela familiaridade que possuo com JVM, que explorarei mais com os benefícios adicionais do Quarkus, como baixo consumo de memória, o pouco tempo de inicialização, pelo aparente sistema apropriado para construções de APIs REST e pela curva de aprendizado que permite fácil adaptação. Como terei meu primeiro contato com a linguagem Go, achei mais apropriado a escolha do Java + Quarkus.

6. Divisão de Responsabilidades

Serviço Principal	Microserviço
Java + Quarkus	Go
Regras de Negócio	Scraping
Autenticação e Persistência	Consumir APIs de lojas externas
Gestão de Wishlist	Disparo de Alertas

7. Equipe:

- Nathalia Emmily Floripes Castro

8. Integrações:

Neste momento, este projeto não está integrado com o projeto de nenhuma outra disciplina